BPFScript: Compiler-Checked Cross-Boundary Typed DSL for eBPF Applications

Paper ID: 29

Abstract

eBPF has become the standard mechanism for extending Linux with custom packet processing, tracing, and security policies, but its programming model is fragmented. A single application spans kernel code, a userspace loader, and shared maps, yet the relationships among them (types, lifecycle ordering, shared state) are checked only at load time or not at all, leading to late, hard-to-diagnose failures. We observe that these cross-boundary relationships are duplicated concepts a type system can unify. We present BPFScript, a DSL that types maps, program handles, and execution domains in one source, then lowers to standard C through the clang/bpftool/libbpf toolchain. The compiler detects mismatches, such as wrong context types or misordered lifecycle calls, before code generation. Generated projects load, attach, and pass traffic on a real kernel, showing that eBPF's cross-boundary structure can be typed while preserving the existing toolchain.

CCS Concepts: • **Software and its engineering** → **Domain specific languages**; *Static analysis*; • **Computer systems organization** → *Reliability*.

Keywords: eBPF, domain-specific language, static typing, systems

1 Introduction

eBPF lets developers run application-specific logic inside the Linux kernel under a verifier that proves memory safety and bounded execution [21]. It now underpins packet processing in major cloud providers, powers observability platforms, enforces security policies, and enables scheduler experimentation within a large and growing ecosystem.

Yet the programming model is fragmented. A single eBPF application spans kernel code constrained by the verifier, a userspace loader that opens objects and manages lifecycle, shared maps and ring buffers, and (for newer features) generated skeleton headers that must match the running kernel. The relationships among these pieces, such as a map's key and value types or the order of load, attach, and detach, are checked only at load time or not at all. Errors surface late as cryptic verifier rejections or silent mismatches.

Existing tools reduce boilerplate yet leave cross-boundary relationships implicit. The standard eBPF development workflow requires hand-written kernel C, userspace C, and a Makefile [14]. Aya compiles both sides from Rust and shares typed maps [1, 3], though lifecycle ordering remains a runtime property. bpftrace targets tracing with higher-level abstractions [2], not general eBPF applications. No existing tool types the full cross-boundary structure at compile time.

The key observation is that these relationships are single concepts duplicated across components. A map declaration is at once a kernel data structure, a userspace lookup target, and a type contract. A program handle is at once a function name, a BPF section, and a file descriptor. In C/libbpf, naming conventions connect those copies, so mismatches surface late. In one typed source, the compiler can see and check them directly.

BPFScript realizes this idea as a compiled, multi-domain DSL. A developer writes eBPF entry points, userspace control flow, maps, and ring buffers in one source file. The compiler types maps, program handles, and execution domains, then lowers to eBPF C, userspace C, and a Makefile. The libbpf and bpftool path produces the final binaries, so the existing toolchain is preserved.

This paper contributes:

- A *typed model of cross-boundary eBPF structure*: execution domains, typed maps, and program-lifecycle handles, each with a typing rule and matching compiler diagnostic (Section 3).
- A compiler that realizes this model while preserving the stock kernel toolchain, so compatibility failures remain visible (Section 4).
- An evaluation showing that each invariant triggers a diagnostic, that one source generates buildable libbpf projects, and that generated objects load, attach, and run on a real kernel (Section 5).

2 Background and Motivation

eBPF allows user-supplied bytecode to run inside the Linux kernel after a verifier proves memory safety and bounded execution [21, 22]. The mechanism now underpins a wide range of applications: tracing and profiling [2, 12], high-performance networking [9], security policy enforcement [19], CPU scheduling [11], page-cache customization [28], GPU resource control [26], and userspace extension runtimes [27]. Every eBPF application has a kernel program and a userspace loader that communicate through maps or ring buffers. The standard eBPF development workflow requires developers to maintain separate kernel C, userspace C, and a Makefile [14]. Aya unifies both sides in Rust [1], and Rex provides safe Rust kernel extensions [13], but these tools still leave cross-boundary relationships implicit.

The kernel/userspace split introduces three kinds of coupling that current tools do not fully address. *Lifecycle coupling* requires operations to occur in a valid order (load before attach, detach before close), but C/libbpf checks this only at runtime. *Representation coupling* requires a map's key and value types to agree across kernel code, generated headers, and userspace lookups, and a mismatch is silent until something fails. *Compatibility coupling* arises because generated bindings depend on the exact kernel and libbpf versions. The verifier detects many errors, though only at load time. A DSL can move lifecycle and representation checks to compile time.

3 A Typed Model of Cross-Boundary Structure

We describe the type system informally because a full semantics or soundness proof is beyond scope. Every rule corresponds to a diagnostic exercised by the rejection suite of Section 5.

A BPFScript program is a flat sequence of declarations. Listing 1 shows the minimal shape of an application, and the rest of this section makes its three relationships precise.

Listing 1. A unified-source eBPF application.

```
include "xdp.kh"

@xdp fn pass_all(ctx: *xdp_md) -> xdp_action {
  return XDP_PASS
}

fn main() -> i32 {
  var prog = load(pass_all)
  attach(prog, "lo", 0)
  detach(prog)
  return 0
}
```

To address representation coupling, each function carries an attribute that assigns it to an execution domain $d \in \{user, ebpf, helper, kfunc\}$. The attributes `@xdp`, `@tc`, `@probe`, `@tracepoint`, and `@perf_event` mark eBPF entry points, while `@helper` marks kernel-shared eBPF code and `@kfunc` marks kernel-module code. An unattributed `fn` is userspace. The checker tags each function with its domain and rejects illegal crossings, such as a userspace function calling an `@helper`. Each eBPF attribute also fixes a context and return type: for example, `@xdp` requires `ctx:*xdp_md` returning `xdp_action`, and `@tc` requires `*__sk_buff` returning `i32`. The compiler rejects any signature that disagrees with its attribute before code generation, so the same surface syntax remains domain-aware: userspace can allocate and print freely, while eBPF code must obey stack and helper restrictions. struct_ops callbacks fit the same scheme, with context and return types determined by the registered slot.

To address representation coupling for shared state, a map is declared as a typed global (`var m : hash<K,V>(N)`) rather than a C section plus separate userspace lookup code. The compiler checks element types at every access site on both sides of the kernel boundary: an index `m[k]` requires `k:K` and yields `V`, in eBPF and userspace alike. A wrong key type, wrong value type, or access to an undeclared map is a compile-time error. One declaration thus replaces kernel definitions, generated fields, and userspace lookups that otherwise must agree by hand.

To address lifecycle coupling, the type system distinguishes a `FunctionRef` (a reference to an `@`-attributed function) from a `ProgramHandle` (a loaded program). The lifecycle builtins are typed so that only `load` produces a `ProgramHandle`, and the formation rule for `FunctionRef` connects lifecycle to domains: only an eBPF entry-point function inhabits `FunctionRef`, so an `@helper`, `@kfunc`, or userspace function cannot be passed to `load`.

$$\frac{f \text{ has an eBPF entry attribute}}{\Gamma \vdash f : \texttt{FunctionRef}}$$

$$\frac{\Gamma \vdash f : \texttt{FunctionRef}}{\Gamma \vdash \texttt{load}(f) : \texttt{ProgramHandle}}$$

$$\frac{\Gamma \vdash h : \texttt{ProgramHandle}}{\Gamma \vdash \texttt{detach}(h) : \texttt{unit}}$$

$$\frac{\Gamma \vdash h : \texttt{ProgramHandle}}{\Gamma \vdash \texttt{attach}(h, t, fl) : \texttt{u32}}$$

Because `attach` and `detach` consume a `ProgramHandle` and only `load` produces one, "attach before load" is a *type* error, not a runtime failure. Passing a bare `FunctionRef`, an integer, or a string to a lifecycle builtin does not type-check. The typing rules enforce a producer/consumer handle discipline: because only `load` produces a `ProgramHandle`, the system enforces the single ordering constraint $load \prec \{attach, detach\}$ rather than the complete load-to-attach-to-detach protocol. It guarantees that lifecycle builtins receive values of the right kind, but not full path-sensitive properties such as use-after-detach, double-attach, or leaked handles.

The benefit is earlier, localized compiler errors in place of naming conventions or late verifier/attach failures. Section 5 measures whether those diagnostics trigger; Section 6 explains the limits of that evidence.

4 Design and Implementation

The compiler has three design goals. First, it must detect cross-boundary errors at compile time, before the verifier or runtime sees them. Second, it must preserve the existing Linux BPF toolchain so that type information, portability, and verifier diagnostics remain available. Third, it must generate idiomatic C that developers can inspect and debug.

The type system draws on established ideas: typestate tracks object state in types [20], and linear types enforce single-use resource protocols [5, 10, 23]. BPFScript applies a lightweight form of these ideas, distinguishing producer

and consumer handles and annotating execution domains, but does not attempt full substructural typing.

To meet these goals, the compiler (written in OCaml with dune) parses BPFScript source, resolves imports and includes, builds symbol tables, runs multi-program analysis and type checking, performs safety analysis, builds an IR, and emits C. The compiler comprises nine modules covering maps, codegen, safety, and struct_ops. For an input `foo.ks`, the generated project contains `foo.ebpf.c`, `foo.c`, a `Makefile`, and, when kfuncs are present, `foo.mod.c`. The Makefile invokes bpftool to dump `vmlinux.h`, clang to produce a BPF object, bpftool to generate a skeleton, and gcc to link the userspace loader against libbpf (together with libelf and zlib).

Each typing rule of Section 3 maps to a concrete compiler pass. The compiler attaches a domain to each function as a scope refined by the entry attribute, and the IR records each function's program type so that code generation can dispatch the same source function to the eBPF or userspace backend. Typed maps lower to a single descriptor that records key and value types once, and both backends read that descriptor to emit the eBPF `SEC(".maps")` definition and the userspace accessor (Listing 2). Lifecycle handles are enforced in the front end: only `load` yields a `ProgramHandle`, and `attach` and `detach` require one, so a misordered call fails type checking before code generation. The handle then lowers to a file descriptor via libbpf APIs.

Listing 2. One typed map declaration lowers to both a kernel `SEC(".maps")` definition and a userspace accessor from a single descriptor.

```
// BPFScript source
var counts : hash<u32, u64>(1024)

// generated eBPF C
struct {
  __uint(type, BPF_MAP_TYPE_HASH);
  __uint(max_entries, 1024);
  __type(key, __u32);
  __type(value, __u64);
} counts SEC(".maps");

// generated userspace C: same key/value types
__u32 key = ...; __u64 val;
bpf_map_lookup_elem(counts_fd, &key, &val);
```

The compiler does not replace the verifier, clang, bpftool, or libbpf, so existing diagnostics remain available and any compatibility failure stays visible. For interfaces that move faster than the DSL can stabilize, `include` and `extern` declarations provide a fallback to raw header declarations.

5 Evaluation

The evaluation answers three questions: *(Q1)* does each typed cross-boundary invariant trigger a compile-time diagnostic? *(Q2)* does one source generate a complete, buildable libbpf project, and does unifying it reduce code footprint and change amplification? *(Q3)* does generated code load, attach, and run on a real kernel? All experiments run on Linux 6.15.11-061511-generic with clang 18, bpftool 7.7, and libbpf 1.3.0. We report SLOC counts as nonblank, noncomment lines, excluding generated headers. We include timing numbers as sanity checks rather than performance claims.

5.1 Q1: Compile-Time Diagnostic Coverage

The compiler test suite contains 85 suites and 1095 tests with no failures, covering all major subsystems from lexer to code generation.

The central Q1 result is a suite of 28 targeted programs that exercises the diagnostics of Section 3 directly: one positive control plus 27 expected rejections covering lifecycle misuses, program-signature violations, map type errors, domain crossings, and safety violations. Each case carries an expected diagnostic substring, so incidental rejections do not count as passing. The suite shows that the invariants are implemented and stable.

To ask whether the checks trigger *earlier* than the stock toolchain on the same mistake, we construct 4 hand-chosen test cases, each a realistic mistake injected identically into a working XDP map-counter written in BPFScript and in matched hand-written C/eBPF, and record the earliest stage each toolchain rejects it along a shared ladder (compile < build < verifier < attach < runtime < undetected). Table 1 reports the outcome. BPFScript rejects all 4 at compile time. The decisive case is the cross-boundary one: an `@xdp` entry point handed a `*__sk_buff` context instead of `*xdp_md`. BPFScript rejects it from the program-signature rule of Section 3. In C the program compiles, and because its body never dereferences the context the verifier has no invalid access to flag, so the mistake survives to a silently wrong load. The matched bodies deliberately touch only a map, not a context field; one that did read a context field could instead trip the verifier on an out-of-range access, but still would not be reported as the context-type confusion it is. The other 3 mistakes tie because clang already catches them locally: an undeclared map, a string stored into a `u64` value, and an oversized stack buffer. The result is therefore 1 earlier and 3 tied: typing helps precisely on the cross-boundary relation that local C checking cannot see. Section 6 explains the limits of this small, author-injected set.

5.2 Q2: Project Generation

To test whether BPFScript generates complete projects, we compiled 44 example programs. Of these, 43 compile successfully and 41 produce working libbpf projects. The single compile-time rejection is `safety_demo.ks`, where safety analysis detects 608 bytes of stack use (above the 512-byte eBPF limit) and halts before codegen, catching a verifier-relevant problem early. The two examples that compile but do not build are both struct_ops programs: each produces a valid eBPF object but fails because the bpftool-generated

Table 1. Differential failure-stage probe: earliest stage each toolchain catches the same injected mistake. BPFScript is strictly earlier only on the cross-boundary context-type mistake; the rest are ties clang already catches.

Injected mistake	Coupling	KS	C/libbpf
Wrong context type	signature/domain	compile reject	undetected
Undeclared map	representation	compile reject	compile reject
String into `u64` value	representation	compile reject	compile reject
Oversized stack buffer	safety	compile reject	compile reject

skeleton references a libbpf field absent from the host's older installed headers, a toolchain version skew we isolate later in this section. For successful examples, the compiler emits a median of 472 lines from 31 source lines. Across 11 workloads with hand-written C/libbpf baselines, BPFScript sources total 203 lines versus 1105. Table 2 shows the feature coverage.

Table 2. Example marker coverage. A = attempted, KS = BPFScript compiler success, B = generated project build success.

Feature	A	KS	B	Feature	A	KS	B
XDP	38	37	37	TC	5	5	5
Probe	4	4	4	Tracept.	2	2	2
Perf	2	2	2	Maps	17	16	16
Ringbuf	3	3	3	Dynptr	1	1	1
kfunc	4	4	3	Str._ops	2	2	0
Tail	2	2	2	User	39	39	37

We hand-ported 3 `xdp-tutorial` XDP programs to BPFScript. The ports total 45 lines while the original C/eBPF bodies total 45 lines, showing that BPFScript handles external code at comparable line counts. A source-only scan of 3 public eBPF repositories confirms that the program classes we target appear in real code.

Change amplification. Source-footprint totals show how much code a finished workload occupies; they do not show how many places a small requirement change touches. To measure that coupling directly, we construct 4 matched micro-edits and compare the diff size of one BPFScript source against a hand-written C/libbpf split. The edits are the ones the typed unified source is meant to centralize: changing a map from `array` to `percpu_array`, changing a program from `@xdp` to `@tc`, changing the attach target symbol, and adding ringbuf reporting plus userspace handling. Table 3 reports modified file count, edit-site count, changed LOC, and whether the author must manually keep kernel code, userspace code, or skeleton/section conventions in sync.

Table 3. Change amplification for four matched micro-edits. "Sync" reports which manually synchronized surfaces change: kernel (K), userspace (U), or skeleton/section conventions (S).

Edit	Impl.	Files	Sites	LOC	Sync
Map `array→percpu_array`	KS	1	1	2	-
	C	2	7	19	K/U/S
Program `@xdp→@tc`	KS	1	2	8	-
	C	2	6	30	K/U/S
Attach target symbol	KS	1	2	4	-
	C	2	2	4	K/U
Add ringbuf reporting/handling	KS	1	5	24	-
	C	2	11	89	K/U/S

The structural result is the decisive one: none of the BPFScript edits require manual cross-boundary synchronization, whereas hand-written C/libbpf requires userspace synchronization in 4/4 cases, kernel-side synchronization in 4/4, and skeleton or section-convention synchronization in 3/4. This is the direct evidence that "one typed source" reduces change amplification rather than only total lines. The diff-size counts point the same way: across the 4 edits, BPFScript has medians of 1 modified file, 2 edit sites, and 6 changed lines, against 2, 6, and 24 for the matched C/libbpf versions; with only 4 edits we read these as illustrative of the same coupling the sync surfaces expose, not as a statistical estimate.

Staged cross-boundary growth. Table 4 compares four stages that add progressively richer cross-boundary structure: pass-only XDP, then a counter map, then ring-buffer reporting, and finally a separate struct_ops callback workload included as a higher-complexity cross-domain reference point rather than as the next step of the same program. The external `xdp-tutorial` ports provide supporting evidence for the early XDP stages only: the 3 pinned programs `basic01-xdp-pass` through `basic03-map-counter` total 45 BPFScript lines and 45 original kernel-C lines. Table 4 itself reports a local matched comparison. Within the XDP progression, BPFScript grows by 2 and 12 lines, while the matched C/libbpf baseline grows by 11 and 202. The separate struct_ops reference point adds 2 lines on the BPFScript side and 142 in the matched C/libbpf baseline. By the reporting stage the maintained baseline is 9.2x the BPFScript source size, and the struct_ops reference point reaches 13.9x. The abstraction-fit result is therefore limited but direct: within one XDP family and again at a more complex cross-domain reference point, the DSL mostly extends one file, while C/libbpf accumulates more coordination code and userspace surface.

Table 4. Staged cross-boundary growth: maintained source footprint for a local matched XDP progression plus a separate struct_ops reference point. C totals include hand-written userspace when the feature requires it.

Stage	KS	C eBPF	C user	C total
Pass-only XDP	10	7	0	7
Add counter map	12	18	0	18
Add ringbuf reporting	24	39	181	220
Shift to struct_ops callbacks	26	52	310	362

5.3 Q3: Runtime Validation

We test a progression of increasingly stringent checks: build success, kernel acceptance, attachability, and runtime behavior. Table 5 summarizes the results on a real kernel.

Table 5. Real-kernel checks: compile, build, verifier-load, and attach.

Stage	Population	Pass
BPFScript compile	44	43
Generated project build	44	41
Verifier-load (strict)	41	37
Isolated XDP attach/detach	27	27

Most generated objects pass the verifier: 37 of 41 from fully built projects load successfully. The 4 failures are a reference-ownership leak, a map-creation failure, a missing kfunc symbol, and an object with no program section. For XDP, 27 of 27 objects attach and detach successfully; the workloads below exercise TC and struct_ops.

Beyond load and attach, matched workloads confirm the generated objects *run*. Under `BPF_PROG_TEST_RUN`, a minimal generated XDP pass program matches its hand-written baseline in instruction count and latency. Over 10 veth/iperf3 trials, generated and hand-written XDP pass programs deliver comparable throughput, and matched TC programs pass traffic as well.

Preserving the kernel toolchain means BPFScript also inherits its compatibility coupling. The two struct_ops build failures are not parser or type errors: both compile to valid eBPF objects but fail because the bpftool-generated skeleton references a libbpf field absent from the installed headers, reflecting mismatched toolchain versions rather than an inherent limitation. A direct libbpf runner (bypassing skeletons) loads both the generated and a matched hand-written tcp-congestion struct_ops object successfully. The failure is a version-skew problem in the generated *skeleton*, exactly the compatibility coupling of Section 2 that source unification does not eliminate.

6 Limitations

The differential probe injects only 4 hand-chosen mistakes; a stronger study would mine real eBPF bugs from revision histories. Lifecycle typing enforces a producer/consumer handle distinction but not full typestate, so use-after-detach and leaked handles are not statically ruled out. The corpus is author-written; the external port shows the line-count advantage does not transfer unchanged. The change-amplification study uses small checked-in micro-edits rather than real revision histories, so it measures maintenance-surface coupling rather than true developer time or debugging effort. Traffic numbers are local sanity checks, not performance claims. Finally, the struct_ops build failures reflect bpftool/libbpf version skew rather than a deep limitation, but they show that generated builds depend on toolchain alignment.

7 Related Work

Prior work on eBPF development falls into three categories. Domain-specific DSLs target particular use cases: bpftrace for tracing [2], BPFBox and ActPlane for security policies [4, 7], BMC for caching [8], P4c-eBPF for networking [16], and Yaksha-Prashna for bytecode analysis [18]. These DSLs generate only kernel code and do not address the cross-boundary structure that BPFScript targets. General eBPF development tools include the standard eBPF development workflow [14], Aya and Rex for Rust [1, 13], Wasm-bpf for WebAssembly deployment [25], and eunomia-bpf for dynamic loading [6]. These reduce boilerplate but leave lifecycle and representation relationships implicit. Verified approaches take a different direction: BeePL generates formally verified bytecode [17], and Jitk and Serval verify JIT correctness [15, 24]. BPFScript occupies a middle ground: it generates both kernel and userspace code from one source and types cross-boundary relationships, but targets practical integration with the existing toolchain rather than formal verification.

8 Conclusion

BPFScript treats the cross-boundary structure of an eBPF application (program lifecycle, shared maps, and execution domains) as something a compiler can type rather than something developers hold together by convention and discover at verifier-load or attach time. The evaluation confirms that each typed invariant triggers a compile-time diagnostic, that one source generates buildable libbpf projects, and that generated objects load, attach, and pass traffic on a real kernel. The result is a concrete design point: typing eBPF's cross-boundary structure is feasible, and because the compiler lowers through the stock toolchain, both what the type system provides and the version-skew limits it cannot absorb stay visible and measurable.

References

[1] Aya contributors. 2026. Aya: Rust eBPF library. https://github.com/aya-rs/aya.

[2] bpftrace developers. 2026. bpftrace: High-level tracing language for Linux eBPF. https://github.com/bpftrace/bpftrace.

[3] Cilium contributors. 2026. cilium/ebpf: eBPF library for Go. https://github.com/cilium/ebpf.

[4] Manuel Costa and Boris Köpf. 2025. Securing AI Agents with Information-Flow Control. *arXiv preprint arXiv:2505.23643* (2025).

[5] Robert DeLine and Manuel Fähndrich. 2001. Enforcing High-Level Protocols in Low-Level Software. In *Proceedings of the ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*. 59–69.

[6] eunomia-bpf developers. 2024. eunomia-bpf: A Dynamic Loading Framework for eBPF. https://github.com/eunomia-bpf/eunomia-bpf.

[7] William Findlay, Anil Somayaji, and David Barrera. 2020. BPFBox: Simple Precise Process Confinement with eBPF. In *Proceedings of the 2020 ACM SIGSAC Conference on Cloud Computing Security Workshop (CCSW)*. 91–103.

[8] Yoann Ghigoff, Julien Sopena, Kenan Music, Kenan Music, and Gilles Muller. 2021. BMC: Accelerating Memcached using Safe In-kernel Caching and Pre-stack Processing. In *Proceedings of the 18th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*. 487–501.

[9] Toke Høiland-Jørgensen, Jesper Dangaard Brouer, Daniel Borkmann, John Fastabend, Tom Herbert, David Ahern, and David Miller. 2018. The eXpress Data Path: Fast Programmable Packet Processing in the Operating System Kernel. In *Proceedings of the 14th International Conference on Emerging Networking Experiments and Technologies (CoNEXT)*. 54–66.

[10] Kohei Honda, Vasco T. Vasconcelos, and Makoto Kubo. 1998. Language Primitives and Type Discipline for Structured Communication-Based Programming. In *Programming Languages and Systems (ESOP)*. 122–138.

[11] Jack Tigar Humphries, Neel Natu, Ashwin Chaugule, Ofir Weisse, Barret Rhoden, Josh Don, Luigi Rizzo, Oleg Noskov, Paul Turner, and Christos Kozyrakis. 2021. ghOSt: Fast & Flexible User-Space Delegation of Linux Scheduling. In *Proceedings of the 28th ACM Symposium on Operating Systems Principles (SOSP)*. 588–604.

[12] iovisor/bcc developers. 2026. BCC: Tools for BPF-based Linux IO analysis, networking, monitoring, and more. https://github.com/iovisor/bcc.

[13] Jinghao Jia, Ruowen Qin, Milo Craun, Egor Lukiyanov, Ayush Bansal, Minh Phan, Michael V. Le, Hubertus Franke, Hani Jamjoom, Tianyin Xu, and Dan Williams. 2025. Rex: Closing the Language-Verifier Gap with Safe and Usable Kernel Extensions. In *2025 USENIX Annual Technical Conference (USENIX ATC 25)*.

[14] libbpf developers. 2026. libbpf: a BPF CO-RE userspace library. https://github.com/libbpf/libbpf.

[15] Luke Nelson, James Bornholt, Ronghui Gu, Andrew Baumann, Emina Torlak, and Xi Wang. 2019. Scaling Symbolic Evaluation for Automated Verification of Systems Code with Serval. In *Proceedings of the 27th ACM Symposium on Operating Systems Principles (SOSP)*. 225–242.

[16] P4 Language Consortium. 2024. P4c-eBPF: P4 Compiler Backend for eBPF. https://github.com/p4lang/p4c/tree/main/backends/ebpf.

[17] Fabian Ruffy, Sophia Pirelli, et al. 2025. BeePL: Correct-by-Compilation Kernel Extensions. In *Principles of Secure Compilation (PriSC @ POPL)*.

[18] Animesh Singh, K. Shiv Kumar, S. VenkataKeerthy, Pragna Mamidipaka, R. V. B. R. N. Aaseesh, Sayandeep Sen, Palanivel A. Kodeswaran, Theophilus A. Benson, Ramakrishna Upadrasta, and Praveen Tammana. 2025. Yaksha-Prashna: Understanding eBPF Bytecode Network Function Behavior. *arXiv preprint arXiv:2602.11232* (2025).

[19] KP Singh et al. [n. d.]. LSM BPF: Kernel Runtime Security Instrumentation. https://docs.kernel.org/bpf/prog_lsm.html. Linux kernel 5.7+.

[20] Robert E. Strom and Shaula Yemini. 1986. Typestate: A Programming Language Concept for Enhancing Software Reliability. *IEEE Transactions on Software Engineering* SE-12, 1 (1986), 157–171.

[21] The Linux Kernel Documentation Project. 2026. eBPF Documentation. https://docs.kernel.org/bpf/.

[22] Marcos A. M. Vieira, Matheus S. Castanho, Racyus D. G. Pacífico, Elerson R. S. Santos, Eduardo P. M. Câmara Juńior, and Luiz F. M. Vieira. 2020. Fast Packet Processing with eBPF and XDP: Concepts, Code, Challenges, and Applications. *Comput. Surveys* 53, 1 (2020), 1–36.

[23] Philip Wadler. 1990. Linear Types Can Change the World! In *Programming Concepts and Methods*, M. Broy and C. Jones (Eds.). North-Holland.

[24] Xi Wang, David Lazar, Nickolai Zeldovich, Adam Chlipala, and Zachary Tatlock. 2014. Jitk: A Trustworthy In-Kernel Interpreter Infrastructure. In *Proceedings of the 11th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*. 33–47.

[25] Yusheng Zheng et al. 2024. Wasm-bpf: Streamlining eBPF Deployment in Cloud Environments with WebAssembly. *arXiv preprint arXiv:2408.04856* (2024).

[26] Yusheng Zheng et al. 2025. gpu_ext: Extensible OS Policies for GPUs via eBPF. *arXiv preprint arXiv:2512.12615* (2025).

[27] Yusheng Zheng, Tong Yu, Yiwei Yang, Yanpeng Hu, Xiaozheng Lai, Dan Williams, and Andi Quinn. 2025. Extending Applications Safely and Efficiently. In *19th USENIX Symposium on Operating Systems Design and Implementation (OSDI 25)*.

[28] Tal Zussman, Ioannis Zarkadas, Jeremy Carin, Andrew Cheng, Hubertus Franke, Jonas Pfefferle, and Asaf Cidon. 2025. cache_ext: Customizing the Page Cache with eBPF. In *Proceedings of the 30th ACM Symposium on Operating Systems Principles (SOSP)*.